



Ejercicio 2 NODE.JS

Desarrollo de una AW con Node.js y MySQL

Objetivo: El objetivo de este ejercicio es desarrollar una aplicación web que se conecte a una base de datos MySQL utilizando **PHPMysqlAdmin** (a través de **XAMPP**) y un **pool de conexiones** en **Node.js**. La aplicación deberá realizar operaciones de inserción, consulta y borrado lógico de datos, y mostrará los resultados en la página web utilizando **response.write**.

Enunciado del Ejercicio:

1. Creación de la Base de Datos:

- Utiliza **PHPMysqlAdmin** (disponible en XAMPP) para crear una base de datos llamada tienda.
- Dentro de la base de datos tienda, crea una tabla llamada productos con los siguientes campos:
 - id (INT, PRIMARY KEY, AUTO_INCREMENT)
 - nombre (VARCHAR(100))
 - precio (DECIMAL(10, 2))
 - disponible (BOOLEAN)
 - fecha_registro (TIMESTAMP)
 - activo (BOOLEAN) - utilizado para el borrado lógico.
- Asegúrate de que la columna "activo" indique si un producto está activo (1) o inactivo (0) para implementar el borrado lógico.

2. Configuración del Pool de Conexiones en Node.js:

- Utiliza el paquete **mysql** en Node.js para configurar un **pool de conexiones** que permita conectar la aplicación a la base de datos tienda.
- El pool de conexiones deberá manejar múltiples peticiones de forma eficiente.
- Define los parámetros de conexión al servidor MySQL (host, usuario, contraseña, base de datos). Los datos serán los siguientes:
 - host:"localhost",
 - user:"root",
 - password:"",
 - database:"tienda"

3. Desarrollo de la Aplicación Web en Node.js:

- Crea una aplicación web con Node.js que realice las siguientes acciones:
 - **Insertar nuevos productos:** Implementa un formulario web que permita insertar un nuevo producto en la tabla productos. El formulario debe solicitar el nombre y precio del producto. La inserción deberá incluir automáticamente la fecha de registro y marcar el producto como activo.



- **Borrado lógico de productos:** Implementa una función que permita marcar un producto como inactivo (borrado lógico) actualizando el campo activo a 0. Este producto no debe aparecer en las consultas posteriores.
- **Consulta de productos activos:** La aplicación deberá mostrar una lista de todos los productos activos (donde activo = 1), visualizados en una tabla HTML.

4. Presentación de Resultados con response.write:

- Utiliza **response.write** para generar las respuestas en el servidor:
 - Tras insertar un nuevo producto, muestra un mensaje como: "Producto [nombre] añadido correctamente."
 - Tras realizar un borrado lógico, muestra un mensaje como: "Producto [nombre] ha sido desactivado correctamente."
 - En las consultas de productos activos, muestra una tabla en HTML con los productos, incluyendo el nombre, precio y fecha de registro.

5. Requisitos Técnicos:

- La aplicación debe usar un **pool de conexiones** para interactuar con la base de datos, garantizando un manejo eficiente de múltiples consultas.
- Implementa el manejo de errores para capturar fallos en la conexión a la base de datos o en las consultas.
- Asegúrate de que los resultados de las operaciones (inserción, borrado lógico, consulta) se muestren dinámicamente en la página web.

6. Estilos y Presentación:

- La tabla de productos y los formularios deberán tener un estilo básico en CSS para mejorar la legibilidad y presentación en la página web.
- La tabla de productos deberá incluir columnas con los campos nombre, precio, disponible, fecha_registro.

Entregables

Los alumnos deberán entregar:

1. **Carpeta del proyecto** con la estructura de archivos completa:
 - Script SQL de la base de datos tienda y de la tabla productos.
 - app.js (servidor con la configuración del pool de conexiones)
 - Cuantos ficheros estáticos que hayan sido utilizados en este ejercicio (css, html, ...)
2. **Archivo** package.json correctamente configurado para que el proyecto pueda ejecutarse sin errores al instalar las dependencias.
3. Capturas de pantalla o demostración de la aplicación web en funcionamiento (inserción, consulta y borrado lógico).

Respecto al nombre del fichero zip se seguirán las normas establecidas en otras entregas, en este caso sería Ejercicio_5_2.zip



Pistas:

- Puedes utilizar la función `mysql.createPool` para crear el pool de conexiones.
- Recuerda liberar las conexiones después de ejecutar una consulta con `connection.release()`.
- Usa la función `response.write()` para generar dinámicamente la página web en respuesta a las operaciones del servidor.